AWS Lambda:
HTTP API Gateway

AWS Lambda

We'll build a **Serverless AI API** using:

- **AWS Lambda** → runs Python code
- **Amazon API Gateway** → exposes HTTP endpoint
- Test using
  - Python → simple AI logic (text processing)
  - POSTMAN
  - Online Tools

Demo Goal

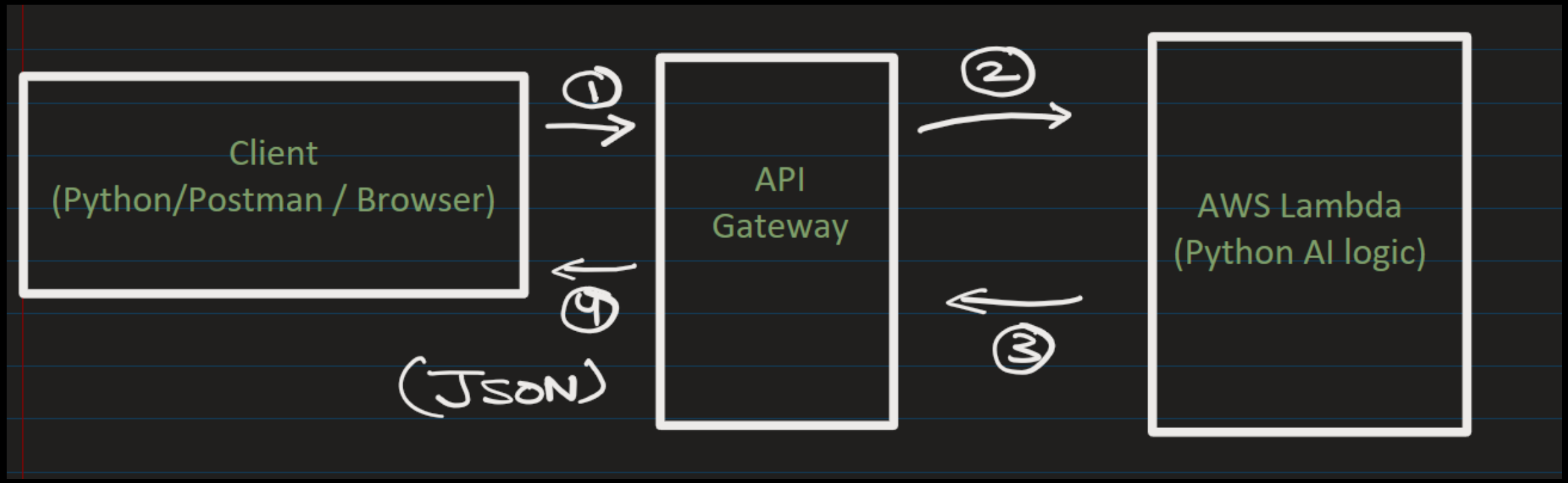
We will call an API like: **https://your-api-url/analyze**

Input: { "text": "I love learning API. I like AWS Lambda because it is easy.“ }

Output: { "result": "Positive Sentiment“ }

We'll use a **simple rule-based AI sentiment analyzer**
(You can later replace it with real ML or OpenAI API)

Architecture



Create Lambda Function

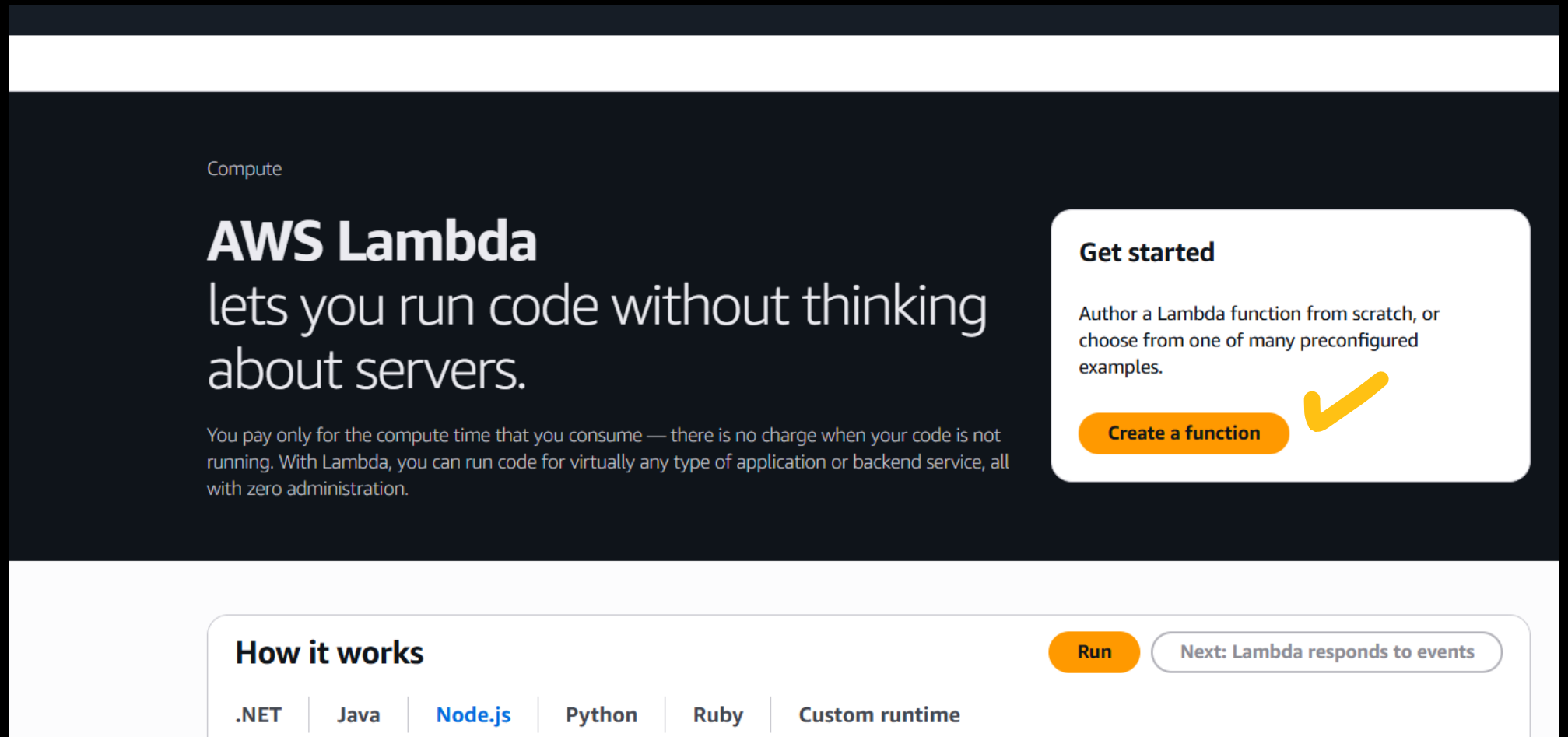
Step 1: Create Lambda Function

Go to AWS → Search **Lambda**
Click: **Create Function**
→ **Author from Scratch**

Fill:

- Function name: ai-sentiment-api
- Runtime: Python 3.x

Click **Create**



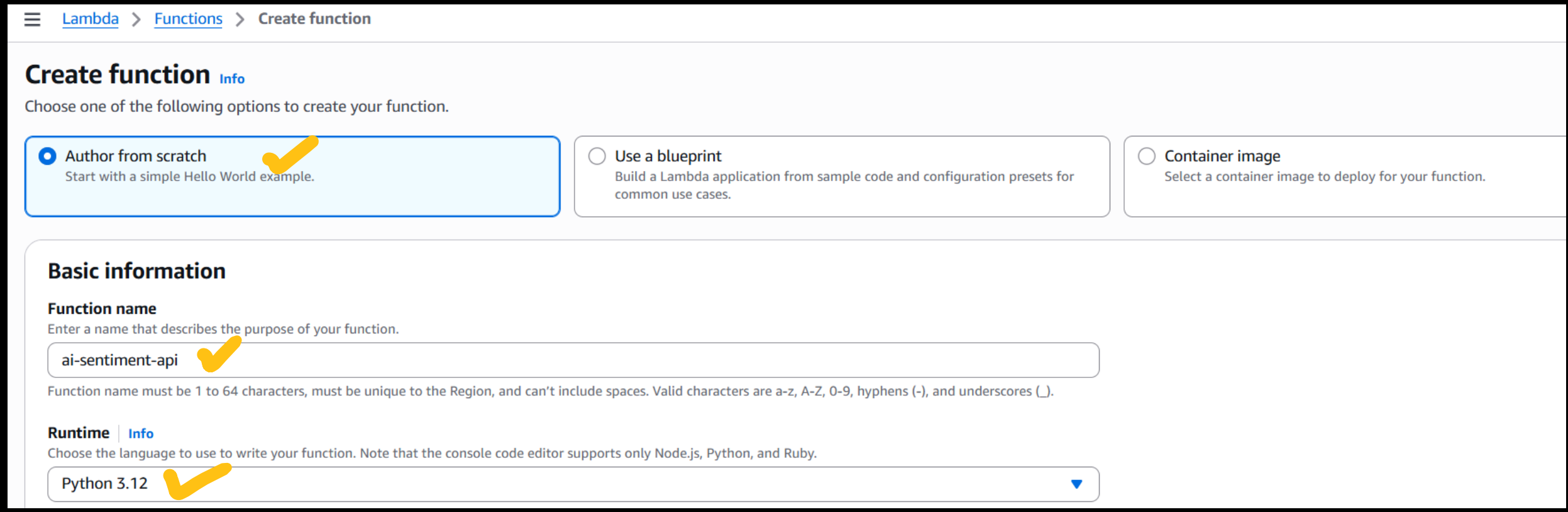
Step 1: Create Lambda Function

Go to AWS → Search **Lambda**
Click: **Create Function**
→ **Author from Scratch**

Fill:

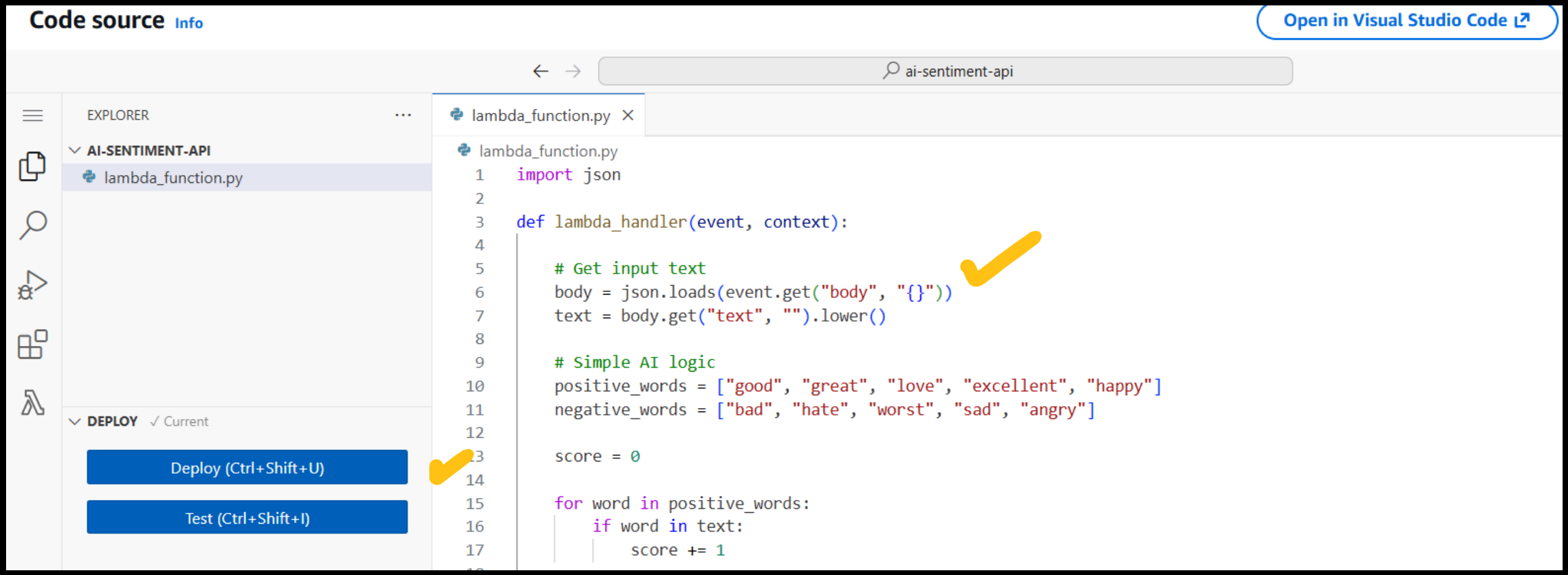
- **Function name: ai-sentiment-api**
- **Runtime: Python 3.x**
- **Architecture: x86_64**

Click **Create**



Step 2: Add AI Code

Replace with this: …. And click **Deploy**



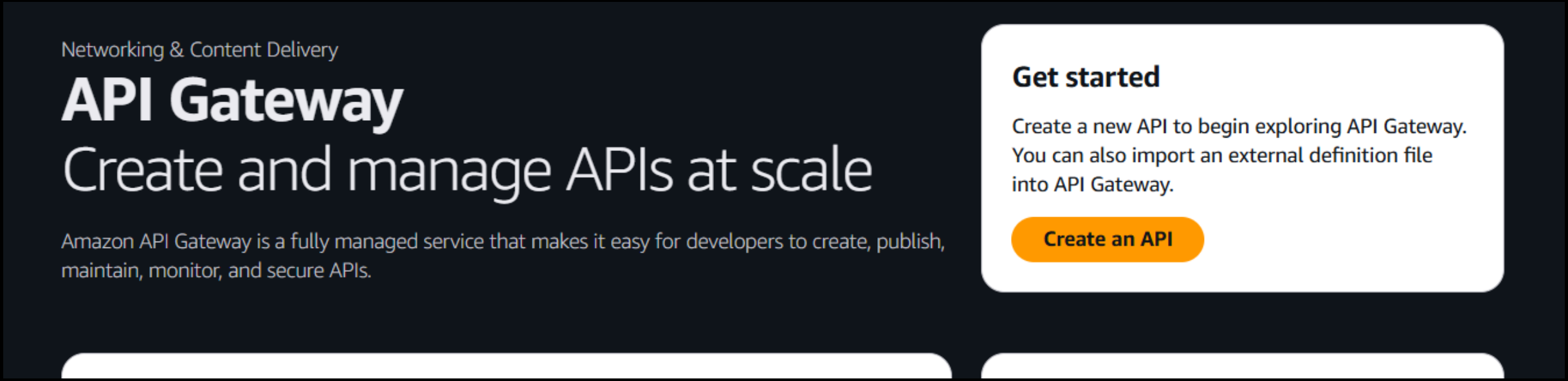
```python
import json

def lambda_handler(event, context):

    # Get input text
    body = json.loads(event.get("body", "{}"))
    text = body.get("text", "").lower()

    # Simple AI logic
    positive_words = ["good", "great", "love", "excellent", "happy"]
    negative_words = ["bad", "hate", "worst", "sad", "angry"]

    score = 0

    for word in positive_words:
        if word in text:
            score += 1

    for word in negative_words:
        if word in text:
            score -= 1

    if score > 0:
        result = "Positive Sentiment "
    elif score < 0:
        result = "Negative Sentiment "
    else:
        result = "Neutral Sentiment "

    return {
        "statusCode": 200,
        "headers": { "Content-Type": "application/json"    },
        "body": json.dumps({   "input": text,  "result": result  })
    }
```

Create API Gateway

Step 3: Create API Gateway

Go to AWS → Search **API Gateway**

Choose **HTTP API** (simpler & faster) and click **Build**



Step 4: Configure API

API-Name: **ai-api**

Add Integration

Select **Lambda** and Choose your function: **ai-sentiment-api**

Configure API

API details

API name

An HTTP API must have a name. The name is a non-unique value you use to identify and organize your APIs. To programmatically refer to this API, use the API ID that API Gateway generates for you.

ai-api

IP address type | Info

Select the type of IP addresses that can invoke the default endpoint for your API. You don't need to redeploy your API for the update to take effect.

- (●) IPv4
  Includes only IPv4 addresses.
- ( ) Dualstack
  Includes IPv4 and IPv6 addresses.

Integrations (1) Info

Specify the backend services that your API will communicate with. These are called integrations. For a Lambda integration, API Gateway invokes the Lambda function and responds with the response from the function. For an HTTP integration, API Gateway sends the request to the URL that you specify and returns the response from the URL.

Lambda

Remove

AWS Region	Lambda function	Version	Learn more
ap-south-1	arn:aws:lambda:ap-south-1:381491992660:function:ai-sentiment-api	2.0	

Add integration

Cancel | Review and create | Next

Add Route:

- Method: **POST**
- Resource path: **/analyze**
- Integration target: **ai-sentiment-api**

Configure routes - *optional*

Configure routes Info

API Gateway uses routes to expose integrations to consumers of your API. Routes for HTTP APIs consist of two parts: an HTTP method and a resource path (e.g., GET /pets). You can define specific HTTP methods for your integration (GET, POST, PUT, PATCH, HEAD, OPTIONS, and DELETE) or use the ANY method to match all methods that you haven't defined on a given resource.

Method	Resource path	Integration target	
POST	/analyze	ai-sentiment-api	Remove

Click: Review and Create (Do Not press Deploy)

Step 1
Configure API

Step 2 - *optional*
Configure routes

Step 3 - *optional*
Define stages

Step 4
Review and create

Review and create

API name and integrations

Edit

API name
ai-api

IP address type
IPv4

Integrations
- ai-sentiment-api (Lambda)

Routes

Edit

Routes

- POST /analyze → ai-sentiment-api (Lambda)

Stages

Edit

Stages

- $default (Auto-deploy: enabled)

Cancel | Previous | Create

API Gateway

APIs
Custom domain names
Domain name access associations
VPC links

API: ai-api(75uk2tajr3)

Develop
Routes
Authorization
Integrations
CORS
Reimport
Export

Successfully created API ai-api (75uk2tajr3).

Routes

Stage: -
Deploy

Routes for ai-api

Create

Search

/analyze
POST

Route details

Delete
Edit

POST /analyze (ID: 2q5jigs)

ARN
arn:aws:apigateway:us-east-1::/apis/75uk2tajr3/routes/2q5jigs

Authorization
Authorizers protect your API against unauthorized requests. Routes with no authorization attached are open.

No authorizer attached to this route.
Attach authorization

Integration
The integration is the backend resource that this route calls when it receives a request.

7bagp93
Configure

Click on the API (LHS)

You'll get an **API endpoint URL**

Example: **https://abc123.execute-api.region.amazonaws.com**

API Gateway

APIs
Custom domain names
Domain name access associations
VPC links

API: ai-api(75uk2tajr3)

Develop
Routes
Authorization
Integrations
CORS
Reimport
Export

Deploy

ai-api

Stage: -
Deploy

API details
Edit

API ID
75uk2tajr3

Protocol
HTTP

Created
2026-03-17

Description
No Description

IP address type
IPv4

Default endpoint
Enabled
https://75uk2tajr3.execute-api.us-east-1.amazonaws.com

ARN
arn:aws:apigateway:us-east-1::/apis/75uk2tajr3

Stages for ai-api (1)

Find resources

Stage name	Invoke URL	Attached deployment	Auto deploy	Last updated
$default	https://75uk2tajr3.execute-api.us-east-1.amazonaws.com	b6z7de	enabled	2026-03-17

Step 5: Test the API – Python Code

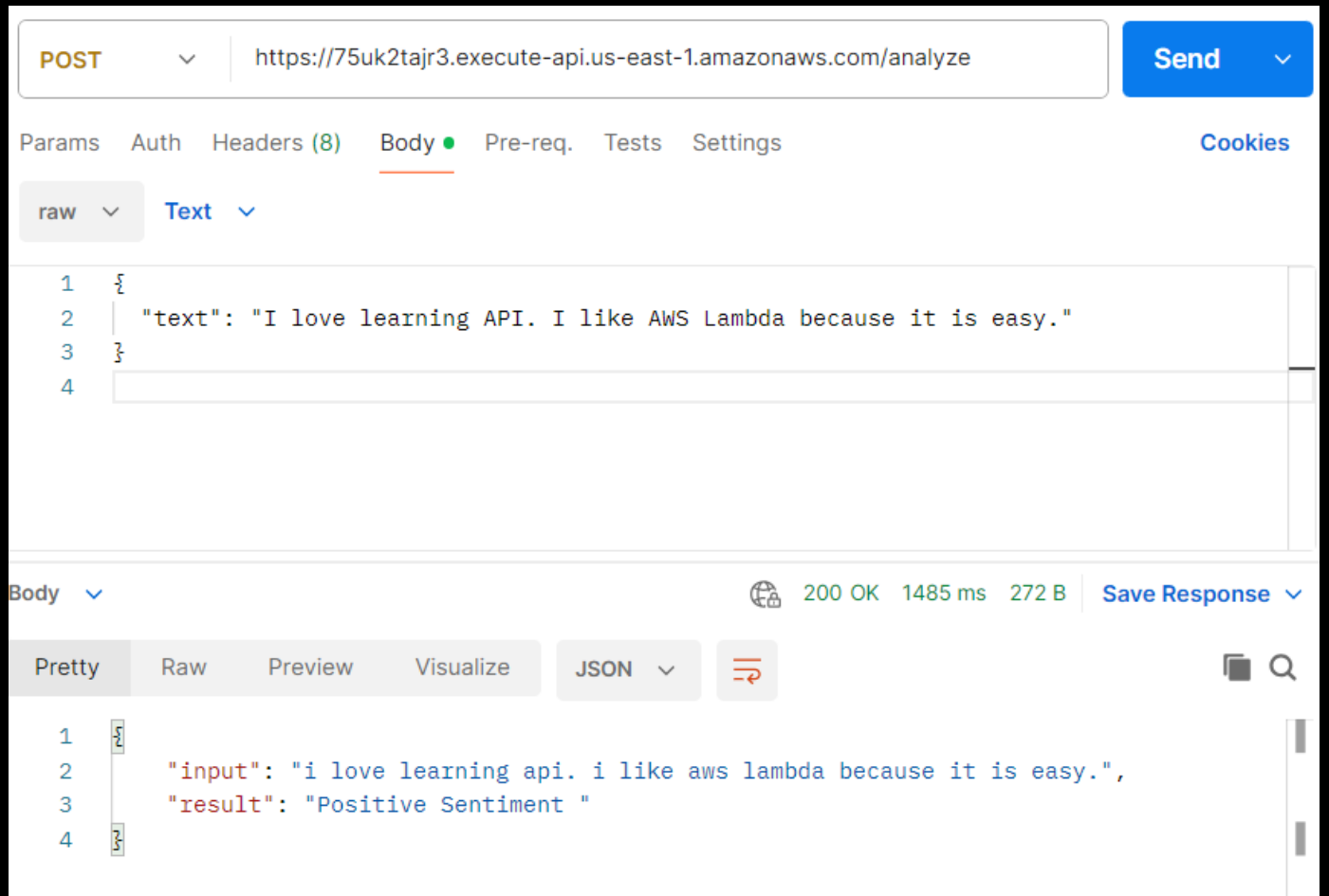
```
import requests
import json

# Replace with your actual API Gateway URL
url = "https://your-api-url/analyze"

payload = { "text": "I love learning API. I like AWS Lambda because it is easy."}
headers = { "Content-Type": "application/json" }

# Make POST request
response = requests.post(url, headers=headers, data=json.dumps(payload))

# Print response
print("Status Code:", response.status_code)
print("Response JSON:", response.json())
```

Step 5: Test the API – POSTMAN

POST request: https://your-api-url/analyze

Body:

```
{
  "text": "I love learning API. I like AWS Lambda because it is easy."
}
```



Step 5: Test the API – www.reqbin.com

POST request: https://your-api-url/analyze

Body:

```
{
  "text": "I love learning API. I like AWS Lambda because it is easy."
}
```

Name: enter name (optional)

Save Share Generate

https://75uk2tajr3.execute-api.us-east-1.amazonaws.com/analyze POST Send

Params Body 4 Auth Headers 6 Raw 11

None JSON Form (url-encoded) XML Custom

```
{
  "text": "I love learning API. I like AWS Lambda because it is easy."
}
```

Body 4 Headers 5 Raw 8 Timings

200 (OK) 346 ms 0.10 kb

Body

```
{
    "input": "i love learning api. i like aws lambda because it is easy.",
    "result": "Positive Sentiment "
}
```

apryse
Web PDF and Office SDK: View, Edi
Customize, Deploy. Free Trial!

Key Points

1. Event-driven execution
Lambda runs only when API is called.

2. Serverless architecture
No AWS EC2. No deployment headache.

3. API creation in minutes
Using API Gateway.

4. AI integration is easy
Even simple logic works.

This is exactly how frontend, mobile app, or ML system talks to a cloud AI service.

MAKE SURE TO DELETE

- API GateWay
- Lambda service

Otherwise, you may be charged

Heading Goes Here

Fhdsklf
Fjdsklf
Fjskldf